Reviewer Pack — Kronecker Coefficient Exponential Gap in Symmetric Powers of…

Entry 722bbadc85c5 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-10 00:22:07 UTC

Kronecker Coefficient Exponential Gap in Symmetric Powers of Permanent vs Determinant

Entry ID: `722bbadc85c5` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-10 00:22:07 UTC

1. Conjecture

Field A (mathematical branch): Representation Theory of Symmetric Groups **Field B** (complexity object): Communication Complexity of Disjointness

Statement:

> For all n ≤ 40, the Kronecker coefficient for the decomposition of $\mathrm{Sym}^{k(\mathrm{Sym}^n(V))}$ into irreducible representations is exponentially larger for the permanent than for the determinant when k = ⌈n/2⌉, implying a Ω(n) communication complexity lower bound for the corresponding disjointness instance.

Rationale (proposer's reasoning):

> Kronecker coefficients capture the structure of symmetric power decompositions, which are central to GCT's orbit closure separations. Linking them to communication complexity provides a bridge between algebraic geometry and combinatorial lower bounds, leveraging the exponential sensitivity of symmetric tensors to combinatorial constraints.

Taxonomy category: `GCT_DET_PERM` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `63c64677514f7f5e`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.95	SAFE	UNCERTAIN

Barrier	Verdict	Confidence	LLM₁	LLM₂
NATURAL_PROOFS	SAFE	0.95	SAFE	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def factorial(n):
        if n == 0 or n == 1:
            return 1
        else:
            return n * factorial(n - 1)

    def kronecker_coefficient(n, k, l):
        if k > n or l > n:
            return 0
        numerator = factorial(n) ** (n - k - l)
        denominator = factorial(k) * factorial(l) * factorial(n - k - l)
        return numerator // denominator

    def symmetric_power_decomposition(n, k):
        coefficients = [kronecker_coefficient(n, i, k - i) for i in range(k + 1)]
        return coefficients

    n = random.randint(5, 40)
    k = math.ceil(n / 2)
    permanent_coefficients = symmetric_power_decomposition(n, k)
    determinant_coefficients = symmetric_power_decomposition(n, k)

    permanent_value = sum(permanent_coefficients)
    determinant_value = sum(determinant_coefficients)

    ratio = permanent_value / determinant_value

    metric_name = "Kronecker Coefficient Ratio"
    metric_value = ratio
    instances_tested = 1
    conjecture_holds = ratio > 2 ** (n - k)
    counterexample = "" if conjecture_holds else f"Ratio {ratio} is not exponentially larger than
```

```
    return {
        "metric_name": metric_name,
        "metric_value": metric_value,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if len(sys.argv) > 1 else [2**i - 1 for i in range(5, 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(result
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        counterexample = next(r["counterexample"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_fa
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
nterexample': 'Ratio 1.0 is not exponentially larger than 2^7'}
TRIAL: {'metric_name': 'Kronecker Coefficient Ratio', 'metric_value': 1.0, 'instances_tested': 1, 'co
TRIAL: {'metric_name': 'Kronecker Coefficient Ratio', 'metric_value': 1.0, 'instances_tested': 1, 'co
TRIAL: {'metric_name': 'Kronecker Coefficient Ratio', 'metric_value': 1.0, 'instances_tested': 1, 'co
TRIAL: {'metric_name': 'Kronecker Coefficient Ratio', 'metric_value': 1.0, 'instances_tested': 1, 'co
TRIAL: {'metric_name': 'Kronecker Coefficient Ratio', 'metric_value': 1.0, 'instances_tested': 1, 'co
TRIAL: {'metric_name': 'Kronecker Coefficient Ratio', 'metric_value': 1.0, 'instances_tested': 1, 'co
TRIAL: {'metric_name': 'Kronecker Coefficient Ratio', 'metric_value': 1.0, 'instances_tested': 1, 'co
TRIAL: {'metric_name': 'Kronecker Coefficient Ratio', 'metric_value': 1.0, 'instances_tested': 1, 'co
TRIAL: {'metric_name': 'Kronecker Coefficient Ratio', 'metric_value': 1.0, 'instances_tested': 1, 'co

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4e40f8cb.py", line 81, in <module>
    first_failing_seed = next(r["seed"] for r in res
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> Test crashed before completing evaluation, but partial results show counterexamples where ratio 1.0 fails to meet exponential thresholds. | next: Fix seed handling in test code and re-run with full coverage of n ≤ 40 cases

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	35610
2	propose	ollama_remote	qwen3:8b	0	0	91838
3	propose	ollama_remote	qwen3:8b	0	0	93058
4	preregistration	ollama_remote	qwen3:8b	0	0	24129
5	novelty	ollama_remote	qwen3:8b	0	0	21158
6	novelty	ollama_remote	qwen3:8b	0	0	13801
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16341
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7809
9	judge	ollama_remote	qwen3:8b	0	0	18125

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 321869 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/722bbadc85c5.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/722bbadc85c5.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/722bbadc85c5.tar.gz` (if generated)